

Resumen Unificado de Scrum

Marco de trabajo, roles, artefactos y eventos

1. Definición, Naturaleza y Objetivo

Scrum no es una metodología ni una ciencia, sino un **marco de trabajo (framework) liviano** que se basa en la inteligencia colectiva. En lugar de brindar instrucciones detalladas o indicar cómo realizar la ingeniería o construcción técnica (a diferencia de procesos como RUP), establece pautas mínimas e innegociables para organizar proyectos, roles, actividades y artefactos. Sus reglas guían las relaciones e interacciones entre las personas, y se tienen que seguir de forma rigurosa para que sea eficiente. Su objetivo es ayudar a personas, equipos u organizaciones a generar valor a través de soluciones adaptables para problemas complejos.

No es una herramienta universal ni genérica, sino que está diseñada específicamente para la gestión y creación de productos complejos en entornos de alta incertidumbre y adaptabilidad. En el trabajo complejo hay más incógnitas que certezas y la relación causa-efecto solo se entiende en retrospectiva, por lo que el equipo necesita tiempo para pensar, rehacer y cambiar de dirección en lugar de seguir planes detallados. Sobre esta base se desarrolla el concepto de "*emergencia*": patrones útiles que surgen de las interacciones del equipo y que no pueden predecirse observando las partes por separado. Scrum no controla la emergencia de forma estricta, sino que la habilita mediante restricciones (eventos con tiempo limitado, roles y bucles de retroalimentación), definiendo eventos a partir de los cuales los equipos van a descubrir las soluciones.

2. Fundamentos Teóricos, Pilares y Pensamientos Complementarios

Scrum se fundamenta en el **Empirismo** (sostiene que el conocimiento proviene de la experiencia y que las decisiones se toman según lo observado) y en el pensamiento **Lean** (se enfoca en reducir el desperdicio orientándose hacia el flujo de valor y la mejora continua). Los 3 pilares empíricos son: **Transparencia** (el proceso y el trabajo deben ser visibles), **Inspección** (los artefactos y el progreso deben evaluarse con frecuencia) y **Adaptación** (ajustar el proceso o producto si se detectan desviaciones).

- **Pensamiento de producto:** las personas consumen productos y no proyectos. Se introduce la tensión entre el corto plazo (tracción o inserción en el mercado masivo) y el largo plazo (deuda técnica, sostenibilidad, moral), planteando que el objetivo del Sprint debe ser siempre un paso hacia el objetivo del producto.

2.1 "Las personas consumen productos y no proyectos"

- **El usuario no ve el backlog:** a un usuario no le importa cuántos tickets cerró el equipo, qué metodología usaron o cuánto duró el sprint. Le importa que la solución le resuelva un problema concreto de forma fluida.

- **Resultado vs. Esfuerzo:** medir el éxito por "cumplir el plan del proyecto" (fechas, alcance fijo, presupuesto) es una trampa si lo que se entrega no genera tracción ni satisface una necesidad real en el mercado.

2.2 La tensión entre el corto y el largo plazo

Desarrollar productos exige hacer un equilibrio continuo entre dos fuerzas que suelen competir:

- **Corto plazo (Tracción / Inserción rápida)**
 - **Objetivo:** salir rápido al mercado para validar hipótesis, ganar usuarios o generar ingresos (lanzar un MVP o nueva funcionalidad).
 - **Riesgo:** si solo se piensa en el corto plazo, se recurre a "atajos". Esto genera **deuda técnica** (código frágil o mal estructurado), problemas de **sostenibilidad** (arquitecturas que no escalan) y **desgaste en la moral del equipo** (frustración por apagar incendios o trabajar con mala calidad).
- **Largo plazo (Sostenibilidad / Calidad)**
 - **Objetivo:** mantener la plataforma estable, escalable y mantenible en el tiempo, asegurando un equipo motivado.
 - **Riesgo:** si solo se piensa en el largo plazo, se cae en la "parálisis por análisis" o la sobreingeniería, tardando demasiado en entregar valor y perdiendo la oportunidad de mercado.

2.3 "El objetivo del Sprint debe ser siempre un paso hacia el objetivo del producto"

En marcos ágiles como Scrum:

- **El Sprint Goal no es una lista de tareas aisladas:** no se trata de meter 10 features descolgadas en dos semanas solo para "ocupar la capacidad" del equipo.
- **Alineación estratégica:** cada Sprint Goal debe responder a la pregunta: ¿Cómo acerca este incremento de software el producto a su visión general?
- **Construcción incremental:** garantiza que las entregas cortas de valor (corto plazo) no sean parches aleatorios, sino escalones estructurados hacia la sostenibilidad y madurez de la solución (largo plazo).
- **Descubrimiento:** proceso continuo de comprender necesidades, enmarcar problemas, generar/priorizar soluciones y validar con evidencia antes de comprometer trabajo de entrega. Debe integrarse con Scrum (visible en el Backlog) y no retrasar la entrega de al menos un incremento por Sprint.

3. Los 5 Valores de Scrum y el Marco OODA

Los valores guían el comportamiento, las decisiones y las acciones del equipo, dando vida a los tres pilares empíricos:

- **Compromiso (Commitment):** el equipo se compromete a alcanzar sus metas (especialmente la Meta del Sprint) y a apoyarse mutuamente.
- **Enfoque (Focus):** su foco principal está en el trabajo del Sprint para lograr el mejor progreso posible.

- **Apertura (Openness):** el equipo y los interesados (stakeholders) son transparentes sobre el trabajo, el avance y las dificultades.
- **Respeto (Respect):** los miembros del equipo se reconocen y respetan como personas capaces e independientes.
- **Coraje / Valentía (Courage):** tienen el valor de hacer lo correcto y abordar problemas difíciles o complejos.

El ciclo OODA

El ciclo OODA (Observar, Orientar, Decidir, Actuar) es un modelo de toma de decisiones para actuar con rapidez en entornos cambiantes e inciertos. No es un proceso lineal que se ejecuta una sola vez, sino un bucle continuo: cada acción genera nueva información que vuelve a alimentar la observación, permitiendo ajustar el rumbo constantemente. Antes de ver cómo los valores de Scrum potencian cada fase, conviene entender qué implica cada una:

- **Observar:** consiste en recolectar información cruda del entorno (datos, señales del mercado, comentarios de usuarios, resultados de un experimento) sin todavía interpretarla ni sacar conclusiones. El objetivo es captar la realidad tal como es, evitando filtrarla con supuestos previos.
- **Orientar:** es la fase más determinante del ciclo: implica interpretar lo observado combinándolo con la experiencia previa, la cultura del equipo y el conocimiento del dominio, para construir una comprensión de la situación. Dos personas pueden observar lo mismo y orientarse de forma distinta según sus supuestos.
- **Decidir:** a partir de esa orientación se elige un curso de acción concreto entre las alternativas posibles. En contextos de incertidumbre no se busca la decisión "perfecta", sino la mejor apuesta disponible con la información actual.
- **Actuar:** es la ejecución de la decisión tomada. El resultado de la acción se convierte en nueva información que reinicia el ciclo desde la Observación, por lo que cuanto más rápido se recorre el bucle, más rápido se aprende y se corrige el rumbo.

Valores que potencian cada fase

Sobre esta base, el Expansion Pack conecta cada fase del ciclo con un valor de Scrum que la potencia, dándoles una función concreta en la toma de decisiones:

- **Observar:** favorecido por la apertura y el respeto para integrar evidencia diversa.
- **Orientar:** el coraje permite orientarse en la incertidumbre y desafiar supuestos.
- **Decidir:** impulsado por el compromiso mediante experimentos pequeños y seguros para fallar.
- **Actuar:** el compromiso sostiene la ejecución dentro de las restricciones del Sprint.

4. El Equipo de Scrum (Scrum Team) y Roles

El equipo debe ser autogestionado, multifuncional, motivado, capacitado, involucrado y comprometido. Se distingue la autogestión de equipo (coordinación fluida) de la individual, y dentro de las dinámicas de

trabajo no existen jefes que asignen tareas. Se incorpora el profesionalismo transversal (responsabilidad sobre el ciclo de vida y excelencia técnica). Todo miembro debe estar enfocado y asignado a un único sprint en un momento dado, con un tamaño ideal de 7 ± 2 personas (o hasta 10 en la extensión). Si alguien no está dispuesto, preparado o capacitado para ejercer su rol como corresponde, debería dejarlo.

Roles básicos

- **Product Owner (PO):** representante del negocio responsable de maximizar el valor del producto y el único que gestiona/ordena el Product Backlog según valor, además de aceptar o rechazar incrementos. Se lo describe más como un líder de producto que como un escritor de requisitos. Jamás debe solaparse con el Scrum Master ni con los Developers.
- **Scrum Master (SM):** responsable de establecer Scrum, vela por la efectividad del marco, remueve impedimentos y guía al equipo y la organización. Podría asumir tareas técnicas de desarrollo en el mismo equipo.
- **Product Developers:** personas comprometidas a crear el Incremento, con autonomía exclusiva sobre cómo transformar ítems en incrementos. El rol pasó de llamarse "Developers" a "Product Developers" para abarcar a creadores de cualquier disciplina, no solo programación de software.

Roles ampliados (Expansion Pack y contexto)

- **Partes implicadas / Stakeholders:** entidad, persona o grupo con interés en el trabajo (clientes, decisores, usuarios, financiadores).
- **Aliado / Supporter:** tipo de parte implicada que actúa como agente de cambio y ayuda a que el equipo prospere, sin ser parte del propio equipo (externo). Son vitales porque un equipo ágil no logra sobrevivir a largo plazo en una estructura u organización que no es ágil sin esta adopción cultural. Ejemplos: compañeros, responsables de decisión, financiadores, responsables de gobernanza, gestores, expertos en la materia, marketing, RR. HH., finanzas, compras y usuarios de piloto.
- **Inteligencia Artificial (IA):** tratada como herramienta de apoyo al control empírico, bajo el compromiso "Human in the loop" (al menos un desarrollador debe ser humano). La responsabilidad sigue siendo humana debido a que la diversidad cognitiva humana aborda la complejidad; por ende, no existe PO IA ni SM IA.

La inteligencia artificial (IA) forma parte del entorno de trabajo y puede ampliar significativamente las capacidades del equipo Scrum en descubrimiento, toma de decisiones, desarrollo de producto y validación de valor.

Los riesgos también son infinitos. Debe mantenerse la responsabilidad humana sobre todos los resultados (según las responsabilidades de Scrum), utilizando la IA como socio supervisado. Esto se conoce como mantener al "*humano en el bucle*". Aunque la IA puede mejorar la innovación y la eficiencia, no sustituye la responsabilidad profesional. Debe apoyar, no anular, el control empírico y la toma de decisiones ética. El equipo Scrum sigue siendo responsable del valor entregado, de analizar la evidencia y de mantener el profesionalismo.

5. Artefactos de Scrum y sus Compromisos

Los artefactos representan trabajo o valor y maximizan la transparencia. Cada uno contiene un compromiso explícito que refuerza el empirismo. Su propósito principal es definir el criterio de éxito y el marco de calidad para medir el progreso de cada artefacto, garantizando que el equipo y los interesados compartan una comprensión clara de lo que se está construyendo.

Product Backlog

Lista emergente y ordenada de todo lo necesario para mejorar el producto; es la única fuente de trabajo. Creado y mantenido por el PO. Puede incluir opcionalmente criterios de aceptación y criterios de resultado. Es dinámico por naturaleza: cambia constantemente para reflejar lo que el producto necesita ser más útil, competitivo y relevante, y nunca se considera completo mientras el producto siga existiendo. Es el PO quien lo ordena según el valor que aporta cada ítem, aunque cualquier persona puede sugerir contenido.

- **Compromiso → Meta del Producto (Product Goal):** describe un estado futuro del producto que sirve como objetivo a largo plazo del equipo. Debe alcanzarse o abandonarse antes de asumir la siguiente. Funciona como un faro que orienta todas las decisiones de priorización dentro del Product Backlog: cada ítem que se selecciona debería representar un paso hacia esa meta, y solo puede perseguirse un Product Goal a la vez.

Sprint Backlog

Plan dinámico (por qué, qué y cómo) creado por y para los Developers, definiendo qué ítems del product backlog se implementarán en ese sprint para lograr la meta o el valor esperado. Es propiedad exclusiva de los Developers: solo ellos pueden modificarlo durante el Sprint, y debe mantenerse visible y actualizado (por ejemplo, revisado en el Daily Scrum) para que el progreso real sea siempre transparente.

- **Compromiso → Meta del Sprint (Sprint Goal):** objetivo único que brinda flexibilidad sobre tareas exactas pero mantiene un foco colectivo. Se define en el Sprint Planning y no cambia durante el Sprint; si el trabajo resulta muy distinto de lo previsto, son los Developers y el PO quienes renegocian el alcance de los ítems, pero nunca la meta en sí, salvo que se decida cancelar el Sprint.

Incremento (Increment)

Un paso concreto hacia la Meta del Producto. Es un entregable utilizable, verificado e integrado con todos los incrementos anteriores. Terminado cada Sprint debería haber un incremento: software funcionando que genere valor. Pueden crearse varios Incrementos dentro de un mismo Sprint, pero siempre deben evaluarse en conjunto con los anteriores, nunca de forma aislada. La decisión de liberar (release) un Incremento al usuario final es independiente de Scrum: el PO puede hacerlo en cualquier momento, no necesariamente al cierre del Sprint.

- **Compromiso → Definición de Terminado (Definition of Done):** descripción formal de los estándares de calidad requeridos para el producto. Si un ítem no cumple con la DoD, no puede considerarse un Incremento ni presentarse en la Sprint Review; en cambio, vuelve al Product Backlog para completarse más adelante (carry over). Es un contrato de calidad compartido por todo el Scrum Team, no una decisión unilateral del PO ni de cada Developer por separado, y suele incluir criterios concretos y verificables (por ejemplo: código revisado por pares, pruebas automatizadas superadas, documentación actualizada, estándares de seguridad o accesibilidad cumplidos). La DoD

puede volverse más exigente con el tiempo a medida que el equipo madura, pero nunca puede relajarse para cumplir con una fecha límite. Se divide en dos dimensiones clave:

- **Output vs Outcome:** el Output es la entrega técnica: el código, la funcionalidad construida, lo que efectivamente se programó y desplegó. El Outcome es el resultado que ese Output produce en el negocio o en el usuario: el problema real que resuelve, el comportamiento que cambia, el valor que efectivamente se captura. Un equipo puede lograr un Output impecable (código sin errores, bien probado) y aun así fracasar si el Outcome es nulo, por ejemplo porque nadie usa esa funcionalidad.
- **División de la DoD:** la DoD del Output se enfoca en la calidad técnica e integración (¿está bien construido, probado e integrado con el resto del sistema?); la DoD del Outcome verifica la generación de valor real (¿resolvió el problema?, ¿se usó?, ¿tuvo el impacto de negocio buscado?). Evaluar solo el Output es una práctica común pero insuficiente: confirma que se entregó algo, no que ese algo sirvió.

6. Eventos de Scrum

Cada evento de Scrum es una oportunidad formal de inspeccionar y adaptar los artefactos. Están específicamente diseñados para permitir la transparencia requerida. El fallo en la operación de cualquiera de estos eventos puede significar oportunidades perdidas para inspeccionar y adaptar. Son usados en Scrum para crear regularidad y reducir reuniones no planeadas.

Todos los eventos son *Time-Box*, teniendo un tiempo máximo fijo definido que no se negocia.

El Sprint

Evento contenedor y "latido del corazón" de Scrum, donde las ideas se convierten en valor, con duración fija de un mes o menos. Todo el trabajo se realiza dentro de él. Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior.

Los Sprints permiten la predictibilidad al garantizar la inspección y adaptación del progreso hacia una Meta del Producto al menos cada mes calendario. Solo el Product Owner tiene la autoridad para cancelar el Sprint.

Durante el Sprint:

- No se realizan cambios que pongan en riesgo la Meta del Sprint.
- La calidad no disminuye.
- El Product Backlog se refina según sea necesario.
- El alcance puede aclararse y renegociarse con el Product Owner a medida que se aprende más.

Sprint Planning

Inicia el Sprint definiendo el trabajo a realizar. Responde a: ¿Por qué es valioso este Sprint? (Meta del Sprint), ¿Qué se puede hacer? (Selección de ítems) y ¿Cómo se realizará el trabajo? (Planificación de tareas).

Estimación y métricas: Capacidad vs. Velocidad

Durante la planificación se profundiza en dos conceptos clave para gestionar la carga de trabajo:

Concepto	Definición	Aplicación
Capacidad	Estimación del volumen de trabajo al que el equipo puede comprometerse. Es el filtro de realidad que usa el equipo durante la <i>Sprint Planning</i> para no comprometerse a más trabajo del que realmente puede ejecutar en ese Sprint específico.	Se mide en horas ideales (equipos nuevos) o puntos de historia (equipos maduros). Determina el compromiso inicial.
Velocidad	Métrica calculada de puntos de historia aceptados por el PO en sprints previos. Una vez estimadas las US, el equipo selecciona del <i>Product Backlog</i> un conjunto de Historias de Usuario cuya suma de SP sea cercana a su velocidad promedio del equipo.	Sirve como referencia empírica basada en trabajo terminado. Prioritaria porque cuantifica el software funcionando. Se mide en SP.

La velocidad se convierte en la métrica prioritaria porque cuantifica directamente ese "software funcionando" que cumple la DoD. No mide intenciones, sino valor técnico e impacto real entregado e inspeccionado en el incremento de software. Mientras que la **Velocidad** mira hacia el pasado (cuántos Story Points *solíamos* hacer en promedio), la **Capacidad** mira el presente (cuántas horas/días de trabajo *reales* tenemos disponibles en las próximas semanas).

Por otro lado, If we sum the story-point estimates for all desired features we come up with a total size estimate for the project. If we know the team's velocity we can divide size by velocity to arrive at an estimated number of iterations. We can turn this duration into a schedule by mapping it onto a calendar. Fortunately, as a team begins making progress through the user stories of a project, their velocity becomes apparent over the first few iterations. The beauty of a points-based approach to estimating is that planning errors are self-correcting because of the application of velocity. Suppose a team estimates a project to include 200 points of work. They initially believe they will be able to complete twenty-five points per iteration, which means they will finish in eight iterations. However, once the project begins, their observed velocity is only twenty. Without re-estimating any work they will have correctly identified that the project will take ten iterations rather than eight

En la estimación relativa, el tamaño en puntos de cada tarea no cambia porque representa la **complejidad/esfuerzo relativo** entre historias (un trabajo de 8 puntos sigue siendo el doble de complejo que uno de 4 puntos). Si el equipo avanza más lento de lo previsto, la fórmula del tiempo total de entrega se ajusta sola sin tocar las estimaciones individuales:

En un enfoque tradicional (Cascada / Estimación Absoluta), habrías estimado que el proyecto requiere, por ejemplo, **1,600 horas de trabajo** para terminarse en 8 semanas (200 horas/semana). Si durante el proyecto ves que no es así, tienes que reestimar el tiempo que te lleva cada tarea para poder volver a calcular el cronograma.

Daily Scrum

Sincronización diaria de los Developers (15 min). Se responde qué hicieron, qué harán y qué impedimentos tienen para adaptar el plan hacia el Sprint Goal. El propósito del Daily Scrum es inspeccionar el progreso hacia la Meta del Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el trabajo planificado próximo.

Sprint Review

El propósito de la Sprint Review es inspeccionar el resultado del Sprint y determinar adaptaciones futuras. Es la inspección del Incremento junto a los stakeholders. Se presentan las historias que cumplen con la DoD; lo no aceptado vuelve al Product Backlog (roll over).

Sprint Retrospective

Inspección y mejora del proceso, personas e interacciones para aumentar la calidad sin buscar culpables. El Scrum Team inspecciona cómo fue el último Sprint respecto a individuos, interacciones, procesos, herramientas y su Definition of Done. Los elementos inspeccionados suelen variar según el dominio de trabajo. Se identifican los supuestos que los llevaron por mal camino y se exploran sus orígenes. El Scrum Team discute qué salió bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no) resueltos. El Scrum Team identifica los cambios más útiles para mejorar su efectividad. Las mejoras de mayor impacto se abordan lo antes posible, incluso pueden agregarse al Sprint Backlog del próximo Sprint.

Refinamiento del Product Backlog

Actividad continua (aprox. 10% del tiempo) para descomponer épicas en historias de usuario del product backlog just in time, asegurando que cumplan la Definition of Ready (DoR). Van preparando las US para que puedan ser incluidas en un próximo sprint.

7. Calidad, Productos con Múltiples Equipos y Modelos

Calidad y cultura

La calidad técnica es innegociable y nunca debe sacrificarse frente a presiones de tiempo. El éxito o fracaso reside en la cultura y el compromiso de las personas.

Múltiples equipos

Escalar no equivale a generar más valor. Si hay varios equipos en un producto, deben compartir el mismo objetivo de producto, el mismo Backlog de Producto y el mismo Propietario del producto.

DoR y DoD

En Scrum, la Definition of Ready (DoR) y la Definition of Done (DoD) son dos listas de criterios acordadas por el equipo que actúan como "filtros de calidad" para garantizar el flujo de trabajo continuo y evitar sorpresas durante el sprint.

La diferencia fundamental está en **cuándo se aplican**: la DoR define cuándo una historia de usuario está lista para *entrar* al sprint, mientras que la DoD define cuándo está terminada para *salir* al mercado o incrementarse al producto.

1. Definition of Ready (DoR) – Criterio de Entrada

Es un conjunto de condiciones que debe cumplir una historia de usuario antes de que el equipo de desarrollo la acepte durante el Sprint Planning. Previene arrancar tareas ambiguas, incompletas o bloqueadas.

- **¿Cuándo se evalúa?:** durante el Product Backlog Refinement (refinamiento) y en el Sprint Planning.
- **Objetivo:** evitar que el equipo comience a trabajar en algo que luego se frene por falta de información o dependencias externas.

Ejemplo típico de DoR:

- ☐ La historia está redactada en formato estándar ("Como [rol], quiero [acción], para [beneficio]").
- ☐ Contiene Criterios de Aceptación claros y medibles.
- ☐ Está estimada por el equipo (p. ej. en Story Points).
- ☐ Las dependencias externas o técnicas están identificadas y resueltas.
- ☐ Los diseños de interfaz (UI/UX) o especificaciones técnicas están adjuntos y aprobados.
- ☐ Es lo suficientemente pequeña como para completarse dentro de un solo sprint (sigue el criterio INVEST).

2. Definition of Done (DoD) – Criterio de Salida

Es un estándar formal que aplica a todo incremento del producto y declara qué condiciones técnicas y de calidad deben cumplirse para considerar que una funcionalidad está **totalmente terminada y lista para producción**.

- **¿Cuándo se evalúa?:** de forma continua durante el Sprint y al momento de mover una historia a Done.
- **Objetivo:** asegurar la calidad técnica, evitar acumular deuda técnica y garantizar un Incremento potencialmente desplegable.

Nota formal según la Guía de Scrum: La DoD es un compromiso oficial del Incremento de Scrum. Si una historia no cumple la DoD, no se puede presentar en la Sprint Review ni contabilizar en la Velocidad del equipo.

Ejemplo típico de DoD:

- ☐ El código está escrito, formateado y subido al repositorio principal mediante Pull Request o Merge.
- ☐ Superó la revisión de código (Peer Code Review) por al menos un compañero.

- ☐ Las pruebas unitarias y de integración están escritas y pasan con éxito (> 80% de cobertura).
- ☐ No existen vulnerabilidades de seguridad críticas detectadas por análisis estático de código.
- ☐ Las pruebas de QA/Aceptación fueron ejecutadas y aprobadas en el entorno de Staging.
- ☐ La documentación interna/técnica (o de API) está actualizada.

Comparación directa

Aspecto	Definition of Ready (DoR)	Definition of Done (DoD)
Enfoque	Preparación y claridad del requerimiento.	Calidad técnica y entrega de valor.
Momento	Antes de iniciar el desarrollo (al entrar al Sprint).	Al finalizar el desarrollo (al cerrar la tarea).
Responsables principales	Product Owner (en colaboración con el equipo).	Equipo de Desarrolladores / QA.
Si no se cumple...	La historia no se planifica en el sprint actual.	La historia no se da por terminada ni suma a la Velocidad.

Product Discovery vs. Product Backlog Refinement

La diferencia principal radica en qué se intenta resolver en cada proceso: mientras el Descubrimiento busca aprender qué problema resolver y qué solución construir, el Refinamiento busca preparar el trabajo validado para que el equipo lo pueda construir sin bloqueos.

1. Product Discovery (Descubrimiento)

- **Objetivo:** reducir la incertidumbre del negocio y del usuario. Es investigar, entender dolores, diseñar prototipos, probar hipótesis y validar si una idea realmente tiene valor y es viable antes de gastar recursos en desarrollarla.
- **Pregunta clave:** *¿Deberíamos construir esto? ¿Aporta valor al usuario y resuelve el problema correcto?*
- **Resultado:** oportunidades validadas con evidencia, ideas descartadas y conceptos claros que luego se convierten en Épicas o Historias de Usuario de alto nivel.

2. Product Backlog Refinement (Refinamiento)

- **Objetivo:** reducir la ambigüedad técnica y de alcance. Es tomar esas soluciones que ya fueron validadas en el Descubrimiento y desmenuzarlas en historias de usuario más pequeñas, redactar criterios de aceptación, estimarlas y verificar que cumplan la Definition of Ready (DoR) para poder meterlas al próximo Sprint.
- **Pregunta clave:** *¿Entendemos exactamente cómo construir esto y está listo para que el equipo lo desarrolle sin frenarse?*

- **Resultado:** historias de usuario pequeñas, estimadas, sin dependencias bloqueantes y listas para el Sprint Planning.

¿Cuándo se valida el Incremento?

El trabajo se valida y acepta **de manera continua a lo largo de todo el Sprint**:

- Conforme los Developers van completando un ítem del Sprint Backlog, lo verifican contra la Definition of Done (DoD).
- El Product Owner colabora diariamente con el equipo y suele revisar/validar las funcionalidades a medida que se terminan durante el Sprint, no al final en una reunión masiva.
- Al llegar a la Sprint Review, lo que no cumple con la DoD simplemente no forma parte del Incremento y no se presenta.